



*Computação em
nuvem*

Azure Service Bus

PAULO R T CÂNDIDO

Azure Service Bus



Motivação Filas – Comunicação Assíncrona

Quando um sistema é dividido em vários microsserviços, fazer com que um serviço chame o outro diretamente via HTTP (comunicações síncronas) o tempo todo cria uma dependência perigosa: se o Serviço B cair ou ficar lento, o Serviço A também cai ou trava.

É aí que entram as filas do Service Bus, atuando como um intermediário (Message Broker).

Cenários Comuns de Uso em Microserviços

1. Desacoplamento Temporal e Resiliência

Se o Microserviço de Pedidos precisa avisar o Microserviço de Faturamento que uma compra foi feita, ele não precisa esperar o Faturamento responder. O serviço de Pedidos simplesmente coloca a mensagem na fila e segue em frente. Mesmo que o Faturamento fique fora do ar por alguns minutos para uma manutenção, as mensagens ficam guardadas em segurança na fila e serão processadas assim que ele voltar.

2. Nivelamento de Carga (Load Leveling)

Imagine uma Black Friday: o microserviço que recebe as vendas sofre um pico gigantesco de requisições. Se ele mandasse tudo direto para o banco de dados ou para um serviço interno pesado, o sistema colapsaria. Com a fila, as mensagens acumulam ordenadamente. O serviço de processamento (os *consumers*) pode ir retirando e processando os dados no seu próprio ritmo, protegendo a infraestrutura de sobrecargas.

3. Garantias de Entrega e "At-Least-Once"

O Azure Service Bus trabalha nativamente com o modelo de **Peek-Lock**. Quando um microserviço pega uma mensagem para processar, ele "tranca" essa mensagem para que ninguém mais mexa nela.

- Se o processamento der certo, o serviço avisa a fila, e mensagem retirada da fila (`Complete`).
- Se o microserviço crashar no meio do caminho, a trava expira e a mensagem volta para a fila automaticamente para ser tentada de novo.

Principais Recursos para Microserviços

Recursos nativos que resolvem problemas complexos de código:

- **Dead-Lettering (Fila de Erros):** Se uma mensagem falhar repetidamente (por erro de código ou dados corrompidos), o Service Bus a move automaticamente para uma sub-fila de mensagens mortas (DLQ). Isso evita que uma mensagem ruim trave o microserviço em um loop infinito.
- **Sessões de Mensagens (FIFO Garantido):** Filas por padrão tentam manter a ordem, mas com o processamento paralelo em microserviços escalados, isso pode se perder. As *Sessions* garantem que mensagens com o mesmo ID de sessão (ex: o mesmo ID de Pedido) sejam processadas estritamente na ordem em que chegaram e pelo mesmo container/instância.
- **Deteção de Duplicidade:** O Service Bus consegue ignorar mensagens repetidas enviadas dentro de uma janela de tempo configurável.

Queue x Topic

Enquanto as **Queues (Filas)** são perfeitas para o modelo *ponto-a-ponto* (um produtor manda, apenas um microserviço específico consome), se você precisar que uma mesma mensagem seja enviada para múltiplos microserviços ao mesmo tempo (modelo *Publish/Subscribe (publicador/assinate)*, como Faturamento e Estoque reagindo ao mesmo pedido), o ideal é evoluir o desenho para usar os **Topics** do Service Bus. Todos os microserviços que assinarem o Tópico receberão a mensagem.

Terraform

Arquivo main.tf em:

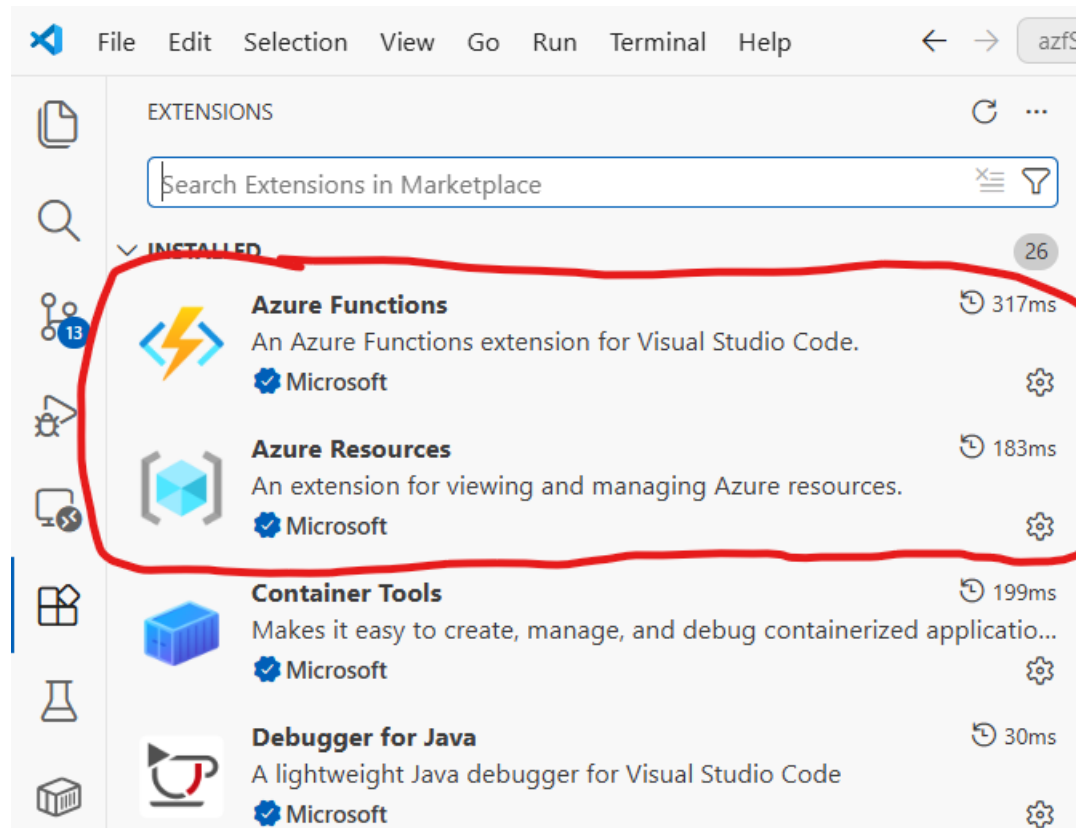
github.com/prtcandido/compnuvem/serviceBus

Após criar a infraestrutura, obter a servicebus_connection_string

```
terraform output -json servicebus_connection_string
```

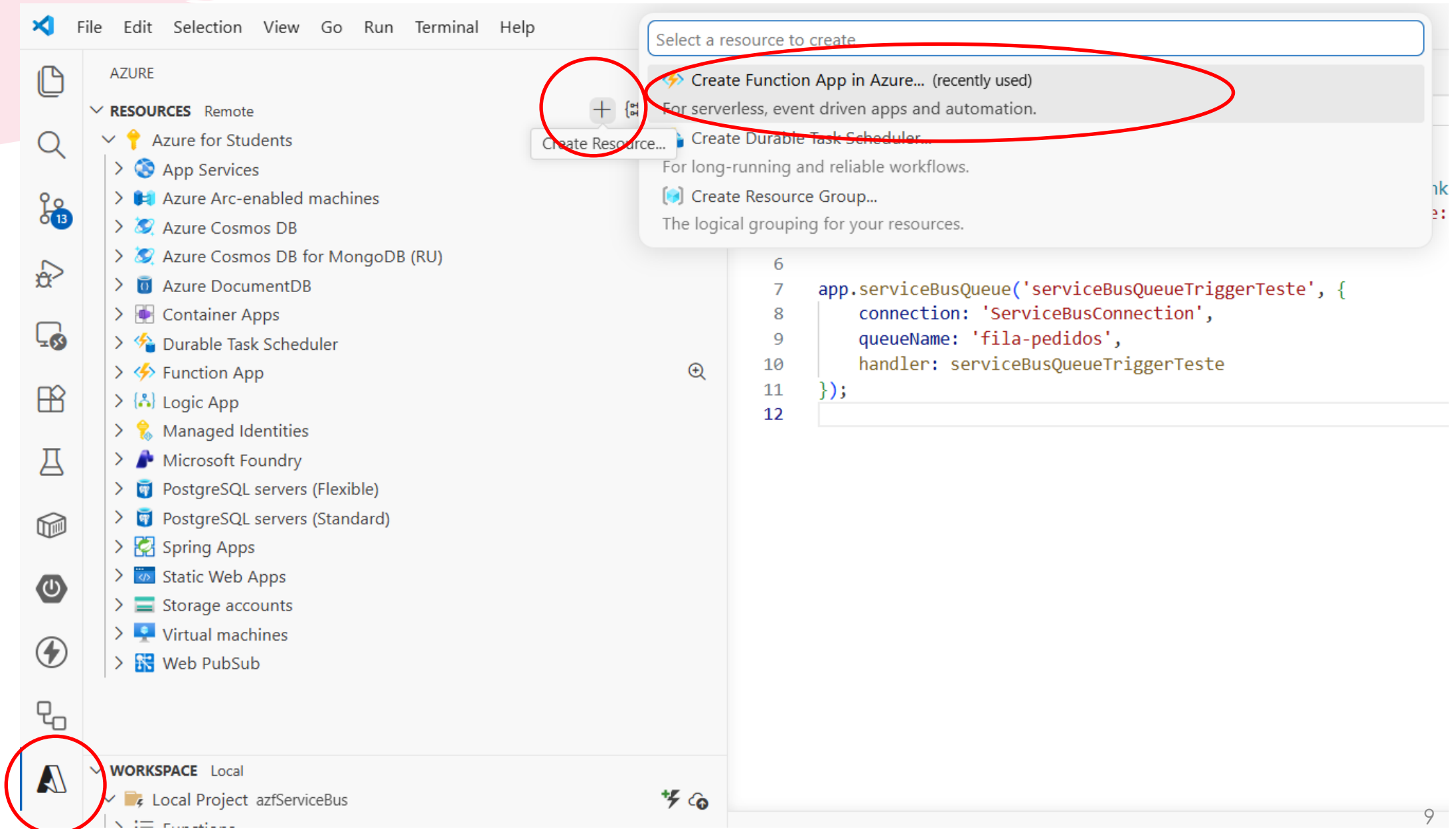

Testar o Service Bus (Azure Functions)

- A. Função HTTP coloca JSON no Azure Service Bus Queue em uma mensagem (App Origem)
- B. Função que consome a mensagem no Azure Service Bus Queue obtendo o JSON oriundo da Função A



Instalar extensões da Azure no Visual Studio Code

Criar Function App (agrupa funções)



Visual Studio Code interface showing the Azure extension. The left sidebar displays the 'AZURE' section with a list of resources under 'Azure for Students'. The 'Function App' resource is highlighted. A red circle highlights the 'Create Resource...' button next to the 'Function App' resource. Another red circle highlights the 'Create Function App in Azure...' option in the dropdown menu that appears when the button is clicked. The dropdown menu also shows options for 'Create Durable Task Scheduler...' and 'Create Resource Group...'. The main editor area shows a code snippet for a Service Bus queue trigger.

```
6
7 app.serviceBusQueue('serviceBusQueueTriggerTeste', {
8     connection: 'ServiceBusConnection',
9     queueName: 'fila-pedidos',
10    handler: serviceBusQueueTriggerTeste
11 });
12
```

Criar as funções

Fonte em:

github.com/prtcandido/compnuvem/serviceBus

The screenshot shows the Visual Studio Code interface with the Azure extension. The left sidebar displays the 'AZURE' section with a list of resources under 'Remote' and 'WORKSPACE Local'. The 'WORKSPACE Local' section is expanded, showing 'Local Project azfServiceBus' and 'Durable Task Scheduler Emulators'. The 'Local Project azfServiceBus' is circled in red. The main editor area shows a TypeScript file named 'httpServiceBusPub.ts' with the following code:

```
1 import { app } from '@azure/functions';
2
3 // 1. Configura o Service Bus
4 const serviceBus = app.serviceBus({
5   queueName: 'queueName',
6   connect: () => {
7     // ...
8   }
9 });
10 export async function httpServiceBusPub(context: FunctionsContext): Promise<void> {
11
12   // 2. Monta o objeto de mensagem esperado pelo binding do Service Bus
13   const pedidoMock = {
14     PedidoId: 102,
15     Cliente: "Professor Teste",
16     Total: 450.00,
17     DataTeste: new Date().toISOString()
18   };
19
20   context.log(`JSON de teste criado: ${JSON.stringify(pedidoMock)}`);
21
22   try {
23     // 3. Monta o objeto de mensagem esperado pelo binding do Service Bus
24     const mensagem = {
25       body: pedidoMock, // O runtime faz o stringify automático para JSON
26       contentType: 'application/json',
27       messageId: `id-teste-${pedidoMock.PedidoId}-${Date.now()}`
28     };
29   } catch (error) {
30     // ...
31   }
32 }
```

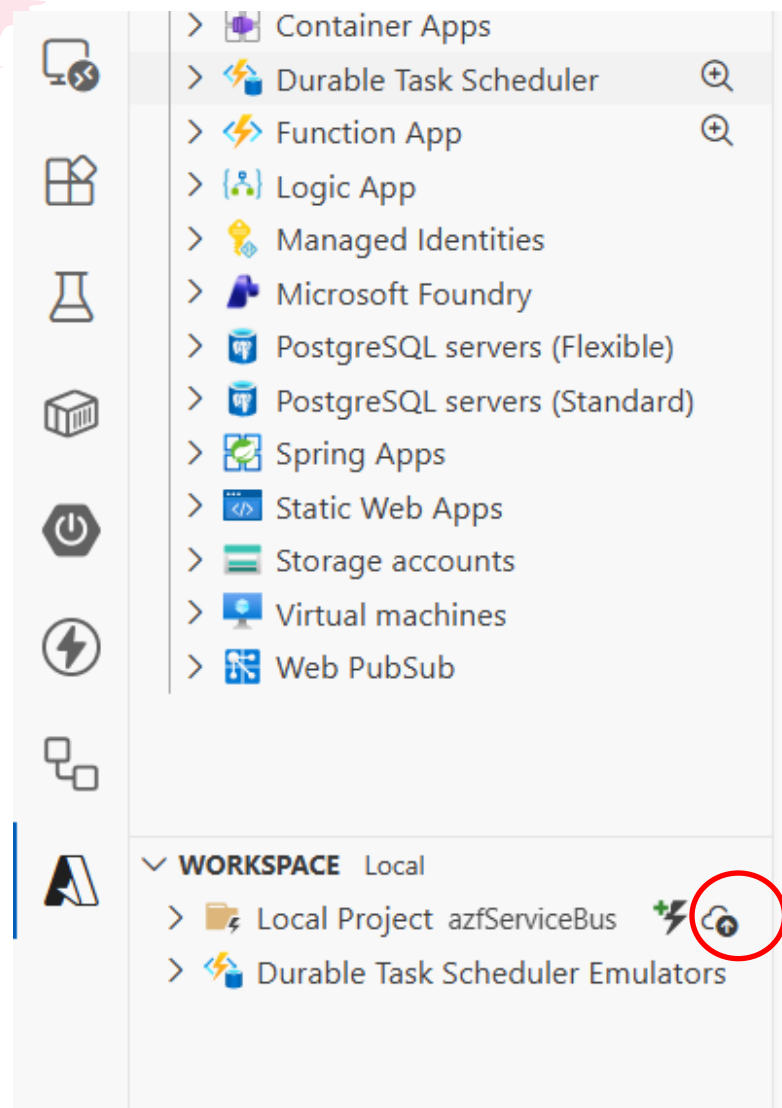
A dialog box titled 'Select a template for your function' is open, showing a list of triggers. The triggers are:

- HTTP trigger (recently used)
- Timer trigger
- Azure Blob Storage trigger
- Azure Blob Storage trigger (using Event Grid)
- Azure Cosmos DB trigger
- Azure Event Grid trigger
- Azure Event Hub trigger
- Azure Queue Storage trigger
- Azure Service Bus Queue trigger
- Azure Service Bus Topic trigger
- Durable Functions entity
- Durable Functions orchestrator
- MCP Tool trigger

Two red boxes with arrows point to the dialog:

- Função A** points to the 'Timer trigger'.
- Função B** points to the 'Azure Service Bus Queue trigger'.

Publicar funções na Azure



8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

```
export async
context.

// 2. Mo
const pe
    Pedi
    Clie
    Tota
    Data
};

context.

try {
    // 3
    cons
};

// 4
cont
```

Quando perguntado,
selecionar o Function
APP criado
inicialmente.

Criar variável de ambiente na Azure

Microsoft Azure

Pesquisar recursos, serviços e documentos (G+/)

Copilot

Página inicial > Resource Manager | Grupos de Recursos > minhasfunc

MinhasFunc
Aplicativo de Funções

Pesquisar

Visão geral

- Log de atividade
- IAM (Controle de acesso)
- Marcações
- Diagnosticar e resolver problemas
- Microsoft Defender para Nuvem
- Eventos (versão prévia)
- Visualizador de recursos
- > Funções
- > Implantação
- > Configurações
 - [x] Variáveis de ambiente**
 - Configuração
 - Durable Functions

Adicione ou remova favoritos pressionando Ctrl+Shift+F

Fundamentos

Grupo de recursos (...): [minhasfunc](#)

Status: Em execução

Localização ([mover](#)): Brazil South

Assinatura ([mover](#)): [Azure for Students](#)

ID da Assinatura: 3c2b3170-57f2-4a04-843f-169f75cea6dd

Rótulos ([editar](#)): [Adicionar marcas](#)

Domínio padrão: [minhasfunc-d](#)

Sistema Operacional: Linux

Tipo de plano: Consumo Flex

Memória da Instância: 2048 MB

Functions | Propriedades | Notificações (0)

{ } Configurar ambiente local [Atualizar](#)

Filtrar por nome...

Nome	Gatilho	Status
httpServiceBusPub	HTTP	✓ Habilitado
serviceBusQueueTriggerTeste	Barramento de Serviço	✓ Habilitado

Criar as funções

Microsoft Azure

Pesquisar recursos, serviços e documentos (G+)

Copilot

Página inicial > Resource Manager | Grupos de Recursos > minhasfunc > MinhasFunc

MinhasFunc | Variáveis de ambiente

Aplicativo de Funções

Pesquisar

Configurações de aplicativo | Cadeias de conexão

Visão geral
Log de atividade
IAM (Controle de acesso)
Marcações
Diagnosticar e resolver problemas
Microsoft Defender para Nuvem
Eventos (versão prévia)
Visualizador de recursos
> Funções
> Implantação
▼ Configurações
Variáveis de ambiente

Pesquisar

+ Adicionar | ↻ Atualizar | 👁 Mostrar valores | ✎ Edição avançada | ↗ V

Nome	Valor	Config
APPLICATIONINSIGHTS_CONNECTION_STRING	👁 Exibir valor	
AzureWebJobsStorage	👁 Exibir valor	
DEPLOYMENT_STORAGE_CONNECTION_STRING	👁 Exibir valor	
ServiceBusConnection	👁 Endpoint=sb://sbns-microservicos-	

Referenciado no código das funções

Ver slide output do Terraform

Verificar funcionamento das funções



The screenshot shows the Microsoft Azure portal interface. At the top, the 'Microsoft Azure' logo is visible. Below it, the breadcrumb navigation path is 'Página Inicial > Resource Manager | Grupos de Recursos > minhasfunc > MinhasFunc'. The main heading is 'httpServiceBusPub | Código + Teste'. Below the heading, there are tabs for 'Código + Teste', 'Integração', 'Teclas de Função', 'Invocações', 'Logs', and 'Métrica'. The 'Código + Teste' tab is selected. Below the tabs, there is a row of action buttons: 'Salvar', 'Descartar', 'Atualizar', 'Testar/Executar', 'Obter URL da função', 'Desabilitar', 'Excluir', and an upward arrow. The 'Obter URL da função' button is circled in red. Below the buttons, the file path 'MinhasFunc / dist / src / functions / httpServiceBusPub.js' is shown. The code editor displays the following JavaScript code:

```
1  "use strict";
2  var __awaiter = (this && this.__awaiter) || function (thisArg, _arguments, P, generator) {
3      ...function adopt(value) { return value instanceof P ? value : new P(function (resolve) {
4      ...return new (P || (P = Promise))(function (resolve, reject) {
5      ...function fulfilled(value) { try { step(generator.next(value)); } catch (e) { reject
6      ...function rejected(value) { try { step(generator["throw"](value)); } catch (e) { r
```

Obter URL e abrir no navegador.

Criar as funções

Microsoft Azure

Pesquisar recursos, serviços e documentos (G+/)

Copilot

paulo.ca
FATEC | CE

Página inicial > Resource Manager | Grupos de Recursos > minhasfunc > MinhasFunc > serviceBusQueueTriggerTeste

serviceBusQueueTriggerTeste | Logs

MinhasFunc

Código + Teste Integração Teclas de Função Invocações **Logs** Métrica

Logs do App Insights ▾ Nível de Log ▾ Parar Copiar Limpar Abrir no Live Metrics Envie-nos seus comentários

```
Conectado!  
2026-05-21T20:23:54 [Information] Executing 'Functions.serviceBusQueueTriggerTeste' (Reason='(null)', Id=7bdf9c35-b676-4065-b341-367170fae533)  
2026-05-21T20:23:54 [Information] Trigger Details: MessageId: affbd954f74544d6ac926d500c7f5216, SequenceNumber: 57, DeliveryCount: 1, EnqueuedTimeUtc: 2026-05-21T20:23:54.4350000+00:00, LockedUntilUtc: 2026-05-21T20:24:54.4350000+00:00, SessionId: (null)  
2026-05-21T20:23:54 [Verbose] Sending invocation id: '7bdf9c35-b676-4065-b341-367170fae533'  
2026-05-21T20:23:54 [Verbose] Posting invocation id:7bdf9c35-b676-4065-b341-367170fae533 on workerId:663d9b4f-5e44-4cf5-9c84-6dfc5683c1c5  
2026-05-21T20:23:54 [Information] Service bus queue function processed message: {  
  body: {  
    PedidoId: 102,  
    Cliente: 'Professor Teste',  
    Total: 450,  
    DataTeste: '2026-05-21T20:23:54.447Z'  
  },  
  contentType: 'application/json',  
  messageId: 'id-teste-102-1779395034447'  
}  
2026-05-21T20:23:54 [Information] Executed 'Functions.serviceBusQueueTriggerTeste' (Succeeded, Id=7bdf9c35-b676-4065-b341-367170fae533, Duration=3ms)
```

Dados da mensagem retirada da Fila pela Função B é apresentada em seu LOG.